

Bài 1:

~~Class~~ Bài 1.

Class là bản thiết kế

Object là những thứ bên trong class.

Có mấy là 1 object mới.

private = chỉ bên trong class đó mới có dùng, tới

public = ai cũng có thể dùng được.

class Bank Account {

acc. balance = 1000;

private String account Number (STK)

owner Name (Tên TK)

balance (số dư)

Bank Account acc = new Bank Account ();

acc. balance = 99; (lỗi)

Chỉ có số dư bên trong class Bank Account mới được
còn được balance

Constructor là điểm báo object sinh ra là
đã dùng được ngay. Tên trùng với class,
không có dấu @ về.

thì chỉ sai hoặc chỉ trùng tên

Bài 2:

Bài 2: Kế thừa Đa hình

cha: Mọi nhân viên đều có (ID, name, baseSalary, displayInfo)

con: Chỉ riêng 1 loại (bonus, workingHours, hourlyRate)

extends: là "kế thừa từ"

protected: nằm ở giữa, bên trong class cho nó cái class con. (cha con đều dùng được)

super = "cha" = gọi constructor của cha

• **new FullTimeEmployee** ("E1", "Long", 100, 50)

1. new cấp vùng nhớ

2. Vào constructor con, nhận 4 tham số

3. gọi **super** (ID, name, baseSalary) → nhảy lên constructor của cha.

4. cha gán **this.ID**, **this.name**, **this.baseSalary**.

5. cha xong → quay về constructor con

6. con gán **this.bonus** = bonus

• **this**: chính object này.

super: phần cha của object này / Lưu là dòng trên

@Override (ghi đè) cùng tên, khác cách làm.

Tham số và kiểu trả về giống cha

báo lỗi ngay lập tức. (khi viết sai method)

Hàm Constructor.

```
class BankAccount {
    private String accountNumber;
    private String ownerName;
    private double balance;
    public BankAccount(String accountNumber, Double balance) {
        this.accountNumber = accountNumber;
    }
}
```

$this.x = x$ (sẽ trừ là field, sẽ trừ là tham số)

Ex:

```
class BankAccount {
    private String accountNumber;
    private double balance;
    public BankAccount(String accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }
}
```

deposit trừ để public và không được để private
vì private chỉ bên trong class mới gọi được, còn
nằm ngoài bị chặn.

Bài 3:

Bài 3:

- **Interface** : danh sách việc phải làm ở ước. Không chứa cách làm. (không có tên ~~no~~ method)
- **implements** : thực hiện danh sách đó.

private PaymentMethod paymentMethod ; ← Interface
(giữ là cái gì đó có thể thanh toán mà không rõ rệt là thẻ hay ví)